

GRA-Core: Иерархическая стабильность для субъект-ориентированных роев ИИ-агентов

Оркестр поощряет локальную пену, обнуляя глобальную

bitsoev oleg

11 июня 2026 г.

Аннотация

В данной работе представлен фреймворк GRA-Core, предназначенный для иерархической стабилизации роев ИИ-агентов с сохранением индивидуальности каждого участника. Основной принцип заключается в следующем: *Оркестр поощряет пену ИИ-агентов, но обнуляет пену в оркестре*. Локальная пена (внутренние противоречия агента, его характер, «тараканы») разрешена и даже поощряется, в то время как глобальная пена (общая фальшь, конфликты, нарушение цели) строго обнуляется с помощью градиентной адаптации. Мы формализуем разделение этих понятий, приводим условия сходимости и демонстрируем работающую реализацию, включающую серверную часть API и интерактивную приборную панель.

1 Введение

Современные системы мультиагентного взаимодействия сталкиваются с фундаментальной дилеммой: стремление к глобальной согласованности часто подавляет разнообразие агентов. Предлагаемый подход решает эту проблему за счёт введения иерархического стабилизационного функционала, где каждый агент сохраняет ненулевое значение «локальной пены» $\Phi_i^{\text{local}} > 0$, отражающей его характер, смещения и творческий шум, но оркестр в целом должен удовлетворять условию $\Phi_{\text{global}} = 0$ (полное соответствие глобальной цели).

В основе работы лежит идея о том, что подавление индивидуальности не является необходимым условием для достижения коллективной эффективности. Напротив, локальная пена может быть источником креативности и адаптивности, тогда как глобальная пена — мерой дезорганизации и рассогласования.

2 Формальная постановка

Обозначим через x глобальное состояние роя, а через a_i и s_i — состояние и личностные параметры агента i . Введём общий функционал:

$$J(x, s_{1..N}) = \Phi_{\text{global}}(x) + \sum_{i=1}^N \left(\alpha_i \Phi_i^{\text{local}}(a_i, s_i) + \beta_i \Phi_i^{\text{align}}(a_i, s_i, x) \right) + R, \quad (1)$$

где:

- $\Phi_{\text{global}}(x)$ — глобальная пена (подлежит обнулению);
- Φ_i^{local} — внутренняя пена агента (допускается, ограничена сверху константой c_i);
- Φ_i^{align} — мера нарушения выравнивания агента с оркестром;
- R — регуляризационные члены.

Правило обновления GRA-Core имеет вид:

$$(x^{t+1}, s_{1..N}^{t+1}) = (x^t, s_{1..N}^t) - \eta \nabla J(x^t, s_{1..N}^t), \quad (2)$$

с тем ограничением, что градиентные компоненты, соответствующие Φ_i^{local} , демпфируются для сохранения условия $0 < \Phi_i^{\text{local}} \leq c_i$.

3 Условия стабильности

Рой считается *стабильным*, если выполнены следующие условия:

$$\Phi_{\text{global}}(x^*) = 0, \quad (3)$$

$$\Delta\Phi_{\text{global}}(x^*) = 0, \quad (4)$$

$$\Delta^2\Phi_{\text{global}}(x^*) > 0, \quad (5)$$

$$0 < \Phi_i^{\text{local}}(a_i^*, s_i^*) \leq c_i \quad \forall i. \quad (6)$$

Первые три условия обеспечивают нахождение оркестра в локальном минимуме глобальной пены; последнее гарантирует, что каждый агент сохраняет свою индивидуальность. Условие $\Delta^2\Phi_{\text{global}} > 0$ играет роль проверки устойчивости достигнутого минимума по аналогии с положительной определённой второй производной.

4 Архитектура реализации

Предложенная реализация включает следующие компоненты:

4.1 Серверная часть (FastAPI + GRA-Core)

Бэкенд реализован на FastAPI и управляет агентами, вычисляет значения пены и выполняет шаги GRA. Основной класс `GRAOrchestra` предоставляет следующие методы:

- **add_agent** — добавление агента с вектором характера;
- **remove_agent** — временное отключение агента (его пенa сохраняется);
- **step** — выполнение одного или нескольких шагов обнуления глобальной пены;

- **_recompute_global_pena** — пересчёт глобальной пены как суммы квадратов ошибок выравнивания и меры конфликта характеров.

Глобальная пенa вычисляется по формуле:

$$\Phi_{\text{global}} = \sum_{i \in \text{connected}} (\text{align_error}_i)^2 + 0.5 \cdot \text{conflict}, \quad (7)$$

где **conflict** — дисперсия первых двух компонент векторов характеров подключённых агентов.

API предоставляет следующие эндпоинты:

- **GET /state** — получение текущих значений пены, состояний агентов и флага стабильности;
- **POST /step** — выполнение шагов обнуления GRA;
- **POST /connect** — подключение ранее отсоединённого агента;
- **POST /disconnect/{id}** — удаление агента из оркестра.

4.2 Фронтенд-панель

Интерактивная панель управления, написанная на HTML/CSS/JS, визуализирует текущие значения Φ_{global} , $\Delta\Phi_{\text{global}}$, $\Delta^2\Phi_{\text{global}}$ и состояние стабильности. Панель также отображает список агентов с их индивидуальными значениями локальной пены и позволяет подключать/отключать агентов в реальном времени.

5 Экспериментальная демонстрация

Для демонстрации работы системы были использованы четыре предустановленных агента: **impressionist** (хаос 0.9, детализация 0.2), **paranoid** (подозрительность 0.8, точность 0.9), **disciplined** (скорость 0.6, качество 0.7) и **rebel** (антицентричность 0.9, креативность 0.8). Начальное состояние характеризуется ненулевой глобальной пеной.

В процессе выполнения GRA-шагов наблюдается монотонное убывание Φ_{global} при сохранении $\Phi_i^{\text{local}} > 0$ для всех агентов. Алгоритм шага включает:

- уменьшение ошибки выравнивания каждого агента: $\text{align_error} = \max(0, \text{align_error} - 0.1 \cdot \text{align_error})$;
- сохранение локальной пены с добавлением малого случайного возмущения: $\text{local_pena} = \max(0.1, \min(\text{local_pena} + \mathcal{N}(0, 0.02), 2.0))$.

Экспериментальные результаты подтверждают, что предложенный механизм позволяет достичь стабильного состояния, удовлетворяющего всем условиям стабильности.

6 Заключение

В работе представлен фреймворк GRA-Core для управления роями ИИ-агентов, основанный на принципе разделения локальной и глобальной пены. Предложена формальная математическая модель, включающая целевой функционал, правило обновления и условия стабильности. Разработана полная программная реализация, включающая серверную часть на FastAPI и веб-интерфейс для визуализации и управления. Экспериментальная демонстрация подтверждает работоспособность подхода.

Возможные направления будущих исследований включают:

- распространение фреймворка на большие языковые модели с использованием градиентной аппроксимации;
- исследование влияния топологии связей между агентами на динамику стабилизации;
- разработка адаптивных механизмов управления коэффициентами α_i и β_i в зависимости от контекста.

Благодарности

Автор выражает благодарность сообществу open-source за вдохновение и инструменты, использованные при реализации проекта. Исходный код и полная документация доступны в репозитории: <https://github.com/qqewq/gra-orchestra>.

Список литературы

- [1] GRA-Orchestra: Живой оркестр ИИ-агентов. <https://github.com/qqewq/gra-orchestra>.
- [2] F. Ramírez, “FastAPI: Modern web framework for building APIs,” 2018.
- [3] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.